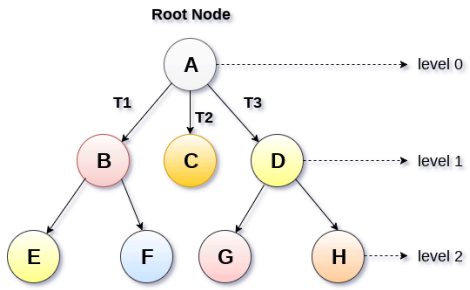
Non-Linear Data Structure – Tree

Unlike in arrays, A Tree stores the data elements in a non-linear fashion. The places where we store the elements in a tree are called nodes. First node of the tree is considered as ROOT of the tree.



Tree

- **Nodes/ Vertices** of the tree stores elements
- **Edges** of a tree connect two nodes in a tree.
- **Root** Node is A.
- **Leaf** Nodes E, F, C, G, H.
- All remaining nodes except root are called **intermediate** nodes.
- B is **parent** node of E and F. And E and F are **child** nodes of B.
- **Left Sub-tree** and **Right Sub-tree**

Basic Terminology

Root Node: The root node is the topmost node in the tree hierarchy. In other words, the root node is the one which doesn't have any parent.

Sub Tree: If the root node is not null, the tree T1, T2 and T3 is called sub-trees of the root node.

Leaf Node: The node of tree, which doesn't have any child node, is called leaf node. Leaf node is the bottom most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes.

Path: The sequence of consecutive edges is called path. In the tree shown in the above image, path to the node E is A→ B → E.

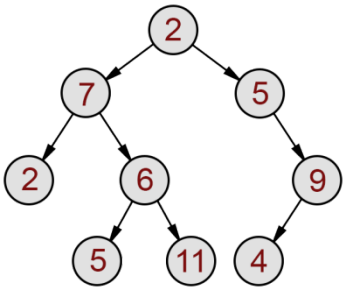
Ancestor node: An ancestor of a node is any predecessor node on a path from root to that node. The root node doesn't have any ancestors. In the tree shown in the above image, the node F have the ancestors, B and A.

Degree: Degree of a node is equal to number of children, a node have. In the tree shown in the above image, the degree of node B is 2. Degree of a leaf node is always 0 while in a complete binary tree, degree of each node is equal to 2.

Level Number: Each node of the tree is assigned a level number in such a way that each node is present at one level higher than its parent. Root node of the tree is always present at level 0.

Binary Tree

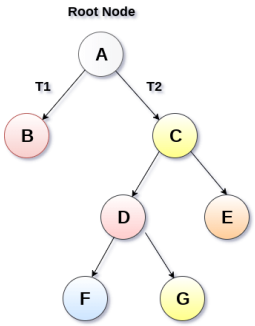
In a tree, if every node contains at most 2 child nodes, that the tree is called binary tree.



TYPES

Strictly Binary Tree

In Strictly Binary Tree, every <u>non-leaf node</u> contains non-empty left and right sub-trees. In other words, the degree of every non-leaf node will always be 2

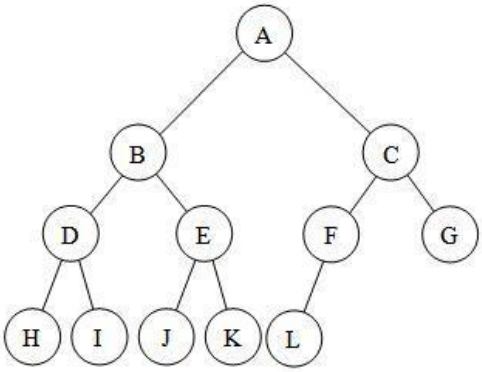


Strictly Binary Tree

Complete Binary Tree

A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible.

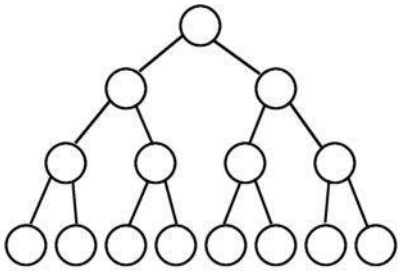
1. All the leaf elements must lean towards the left.
2. The last leaf element might not have a right sibling i.e. <u>a complete binary tree doesn't have to be a full binary tree</u>.



Full Binary Tree

A binary tree of H strictly (or exactly) containing ***2^(H+1) -1 nodes*** , it's clear that which every level has the most nodes. Or in short strict binary trees where all leaf nodes are at same level.

Full Binary Tree



Binary Tree Representation

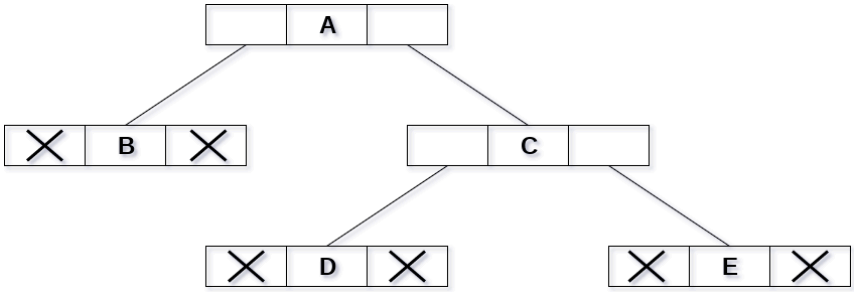
There are two types of representation of a binary tree:

1. Linked Representation

In this representation, the binary tree is stored in the memory, in the form of a linked list where the number of nodes are stored at non-contiguous memory locations and linked together by inheriting parent child relationship like a tree.

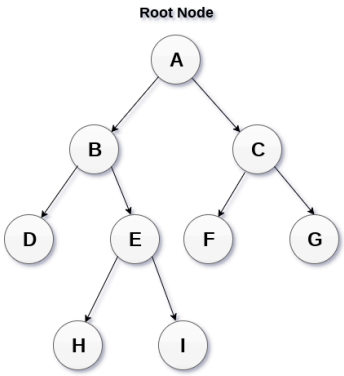
Every node contains three parts:
- Pointer to the left node
- Data element and
- Pointer to the right node.

Each binary tree has a root pointer which points to the root node of the binary tree. In an empty binary tree, the root pointer will point to **NULL**.



2. Sequential Representation (Array Representation)

This is the simplest memory allocation technique to store the tree elements but it is an inefficient technique since it requires a lot of space to store the tree elements. In this representation, an array is used to store the tree elements. Size of the array will be equal to the number of nodes present in the tree. The root node of the tree will be present at the index 0. If a node is stored at i^{th} index then its left and right children will be stored at *2*i+1* and *2*i+2* location.



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

A	B	C	D	E	F	G			H	I				

Binary Tree Traversal

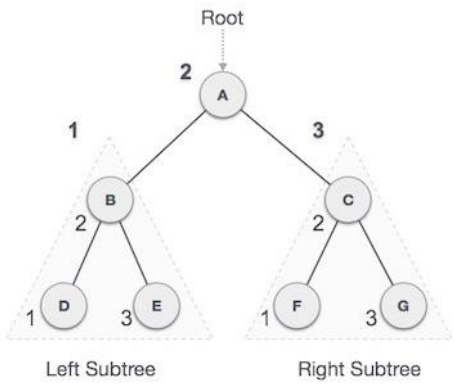
Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree. There are three ways which we use to traverse a tree –

- In-order Traversal
- Pre-order Traversal
- Post-order Traversal

Generally, we traverse a tree to search or locate a given item or key in the tree or to print all the values it contains.

In-order Traversal

In this traversal method, the left subtree is visited first, then the root and later the right sub-tree. We should always remember that every node may represent a subtree itself. If a binary tree is traversed **in-order**, the output will produce sorted key values in an ascending order.



[Left subtree] root [Right subtree]
[Left subtree] B [Right subtree] root [Left subtree] C [Right subtree]

D B E A F C G

We start from **A**, and following in-order traversal, we move to its left subtree **B**. **B** is also traversed in-order. The process goes on until all the nodes are visited. The output of in-order traversal of this tree will be –

D → B → E → A → F → C → G

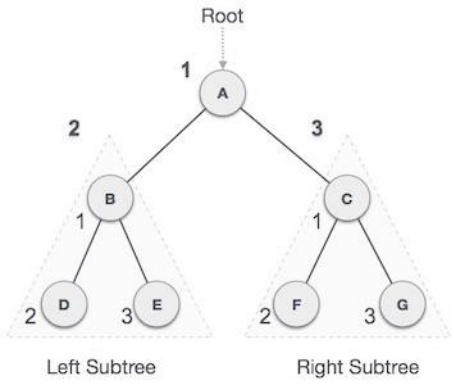
Algorithm

Until all nodes are traversed -
Step 1 - Recursively traverse left subtree.
Step 2 - Visit root node.

Step 3 - Recursively traverse right subtree.

Pre-order Traversal

In this traversal method, the root node is visited first, then the left subtree and finally the right subtree.



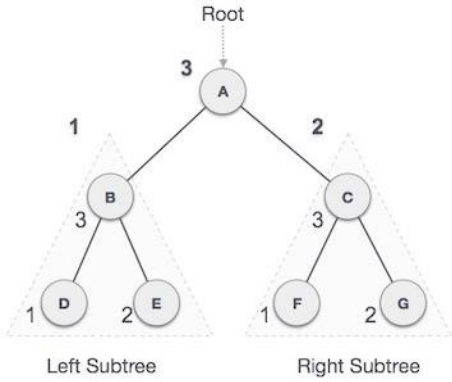
A → B → D → E → C → F → G

Algorithm

Until all nodes are traversed -
Step 1 - Visit root node.
Step 2 - Recursively traverse left subtree.
Step 3 - Recursively traverse right subtree.

Post-order Traversal

In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.



D → E → B → F → G → C → A

Algorithm

Until all nodes are traversed -

Step 1 - Recursively traverse left subtree.
Step 2 - Recursively traverse right subtree.
Step 3 - Visit root node.

Creation of binary tree from in-order and pre/post order traversals

1) Let us consider the below traversals:

In-order sequence: [D **B** E] **A** [F **C]**
Pre-order sequence: **A** B D E C F

In a Preorder sequence, leftmost element is the root of the tree. So we know 'A' is root for given sequences. By searching 'A' in In-order sequence, we can find out all elements on left side of 'A' are in left subtree and elements on right are in right subtree. So we know below structure now.

```
      A
     /  \
D B E    F C
```

We recursively follow above steps and get the following tree.

```
      A
     /  \
    B    C
   / \  /
  D   E F
```

2) Let us consider the below traversals:

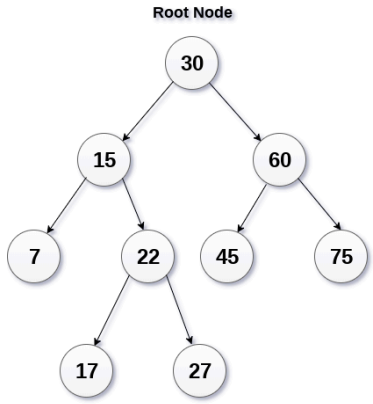
In-order sequence: D **B** E **A** F **C**
Post-order sequence: D E **B** F **C** **A**

```
    A
   /  \
DBE    FC
```

```
      A
     /  \
    B    C
   / \  /
  D   E F
```

Binary Search Tree

1. Binary Search tree can be defined as a class of binary trees, in which the nodes are arranged in a specific order. This is also called ordered binary tree.
2. In a binary search tree, the **value** of all the nodes in the **left sub-tree is less** than the value of the root.
3. Similarly, **value** of all the nodes in the **right sub-tree is greater** than or equal to the value of the root.
4. This rule will be recursively applied to all the left and right sub-trees of the root.



Binary Search Tree

Advantages of Binary Search Tree

1. Searching become very efficient in a binary search tree since, we get a hint at each step, about which sub-tree contains the desired element.
2. The binary search tree is considered as efficient data structure in compare to arrays and linked lists. In searching process, it removes half sub-tree at every step. Searching for an element in a binary search tree takes $O(\log_2 n)$ time. In worst case, the time it takes to search an element is 0(n).
3. It also speeds up the insertion and deletion operations as compare to that in array and linked list.

Operations on Binary Search Tree

There are many operations which can be performed on a Binary Search Tree. Three common operations performed on BST as follows:

Search: Finding the location of a specific element in a Binary Search Tree.

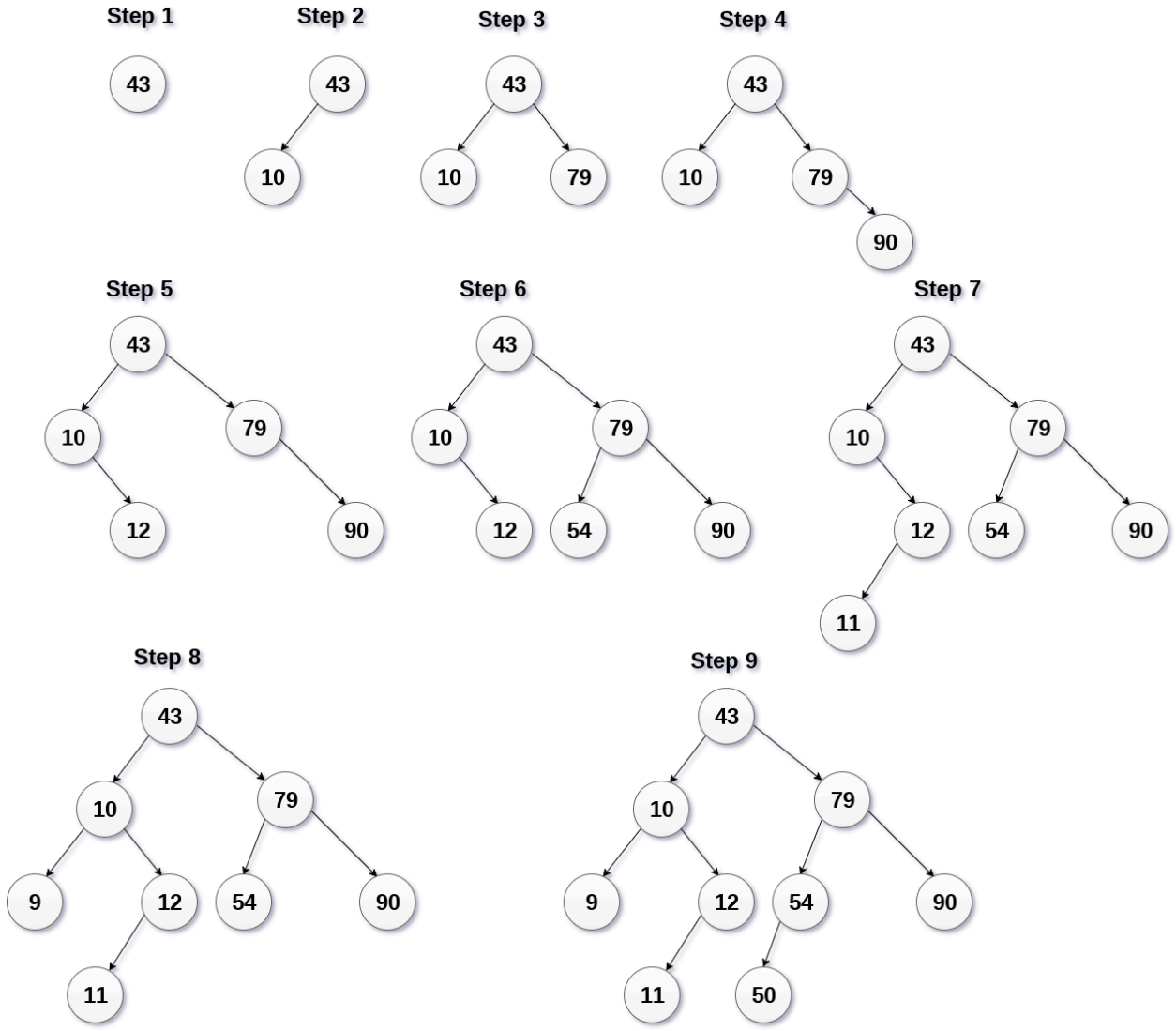
Insert: Adding a new element to the Binary Search Tree at the appropriate.

Delete: Deleting a specific node from a Binary Search Tree.

Construction of Binary Search Tree

The process of creating the Binary Search Tree using the following data elements is shown bellow.

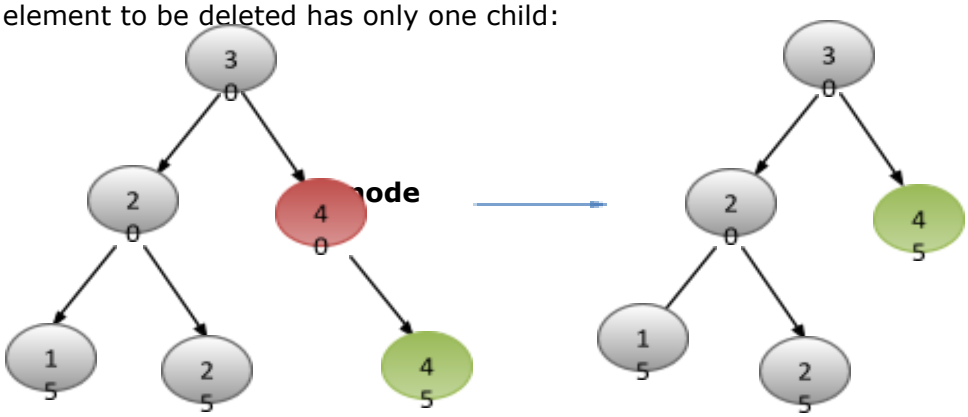
43, 10, 79, 90, 12, 54, 11, 9, 50



Binary search Tree Creation

Deleting a node in Binary Search Tree: Find the element to be deleted recursively

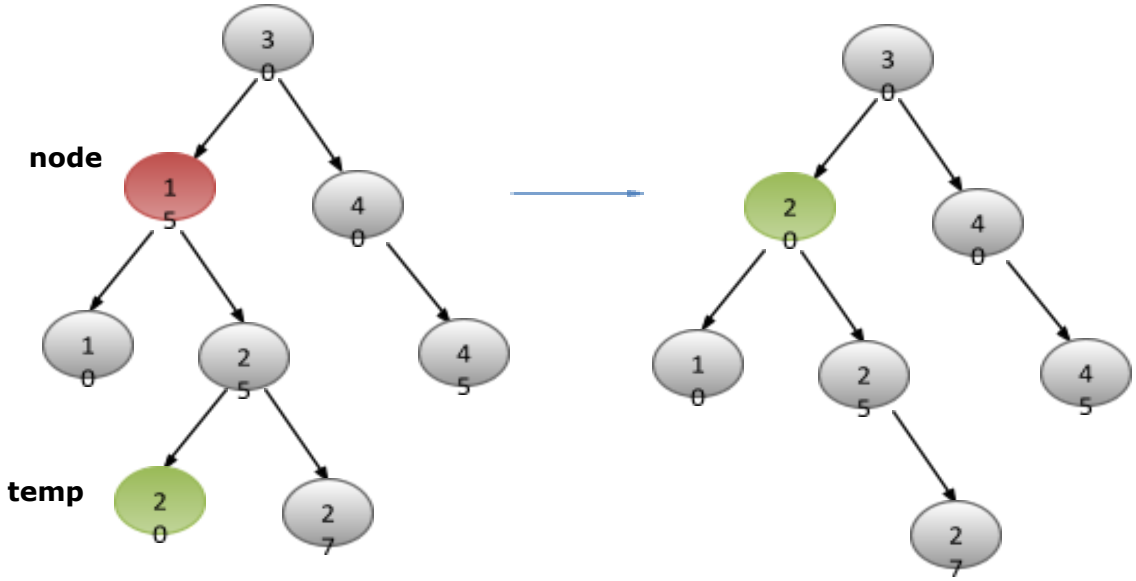
1. If element to be deleted has only one child:



10, 15, 20, 25, 27, 30, 40, 45 **successor**

```
else
{
        temp = node;
        if(node->right == NULL)
                node = node->left;
        else if(node->left == NULL)
                node = node->right;
        free(temp);
}
```

2. If element to be deleted has two child nodes:

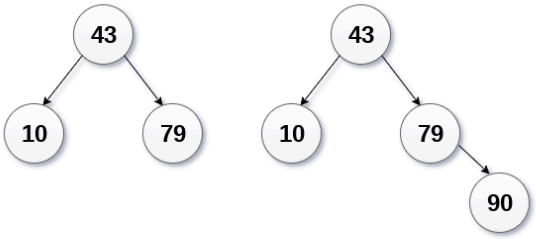



```
if(node->left!=NULL && node->right!=NULL)
{
        temp = findMin(node->right);
        node->data = temp->data;
        node->right = deleteNode(node->right, temp->data);
}
```

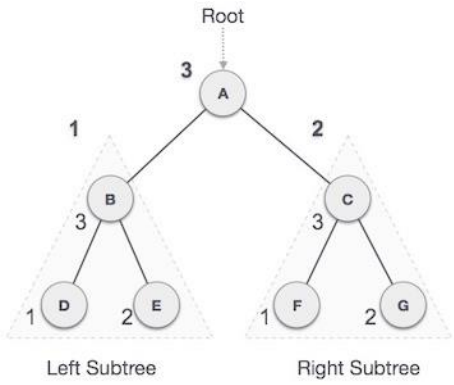
```
#include<stdio.h>
#include<stdlib.h>
typedef struct Node
{
	int data;
	struct Node *left, *right;
} bstNode;

bstNode* insert(bstNode *temp, int val)
{
	if(temp == NULL)
	{
		temp = (bstNode*)malloc(sizeof(bstNode));
		temp->data = val;
		temp->left = temp->right = NULL;
	}
	else if(val == temp->data)
		printf("\nNode already existing\n");
	else if(val < temp->data)
		temp->left = insert(temp->left, val);
	else
		temp->right = insert(temp->right, val);

	return temp;
}
```




```
void inorder(bstNode *root)
{
	if(root != NULL)
	{
		inorder(root->left);
		printf("%d ", root->data);
		inorder(root->right);
	}
}
void preorder(bstNode *root)
{
	if(root != NULL)
	{
		printf("%d ", root->data);
		preorder(root->left);
		preorder(root->right);
	}
}
void postorder(bstNode *root)
{
	if(root != NULL)
	{
		postorder(root->left);
		postorder(root->right);
		printf("%d ", root->data);
	}
}
```



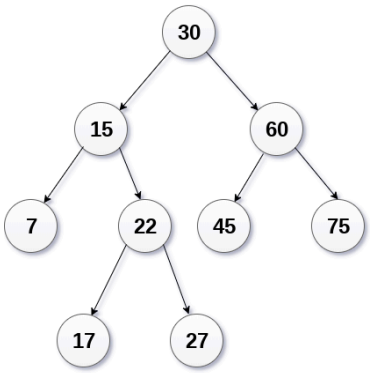
```
bstNode* findMax(bstNode *temp)
{
	if(temp->right != NULL)
		return findMax(temp->right);

	return temp;
}

bstNode* findMin(bstNode *temp)
{
	if(temp->left)
		return findMin(temp->left);

	return temp;
}

bstNode* deleteNode(bstNode *node, int val)
{
	bstNode *temp;
	if(node == NULL)
	{
		printf("\n Node not found");
	}
	else if(val < node->data)
	{
		node->left = deleteNode(node->left, val);
	}
	else if(val > node->data)
	{
		node->right = deleteNode(node->right, val);
	}
	else
	{
		if(node->left!=NULL && node->right!=NULL)
		{
			temp = findMin(node->right);
			node->data = temp->data;
			node->right = deleteNode(node->right, temp->data);
		}
		else
		{
			temp = node;
			if(node->right == NULL)
				node = node->left;
			else if(node->left == NULL)
				node = node->right;
			free(temp);
		}
	}
	return node;
}
```



```
int main()
{
	bstNode *root = NULL, *temp;
	while(1)
	{
		int choice, val;
		printf("\n BST Operations: ");
		printf("\n 1. Insert \n 2. Inorder \n 3. Preorder \n 4. Postorder");
		printf("\n 5. Find Min \n 6. Find Max \n 7. Delete \n 8. Exit ");
		printf("\n Enter your choice: ");
		scanf("%d", &choice);
		switch(choice)
		{
			case 1: printf("\n Enter the value: ");
				scanf("%d", &val);
				root = insert(root, val);
				break;
			case 2: printf("\n Inorder: ");
				inorder(root);
				break;
			case 3: printf("\n Preorder: ");
				preorder(root);
				break;
			case 4: printf("\n Postorder: ");
				postorder(root);
				break;
			case 5: if(root != NULL)
				{
					temp = findMin(root);
					printf("\n Min = %d", temp->data);
				}
				break;
			case 6: if(root != NULL)
				{
					temp = findMax(root);
					printf("\n Max = %d", temp->data);
				}
				break;
			case 7: printf("\n Enter the node to delete: ");
				scanf("%d", &val);
				root = deleteNode(root, val);
				break;
			default:return 0;
		}
	}
}
```